

第一題 Decision-Tree Classifier Tutorial:

調整模型參數:

- max_depth : 避免過度複雜
- min_samples_split 或 min_samples_leaf : 增加樣本數限制

改善前:

使用基尼指數作為準則實例化決策樹分類器模型

```
clf_gini = DecisionTreeClassifier(criterion='gini', max_depth=3,  
random_state=0)
```

使用基尼係數檢定準確率

```
Model accuracy score with criterion gini index: 0.8021
```

比較訓練集和測試集的準確率

```
Training-set accuracy score: 0.7865
```

```
Training set score: 0.7865
```

```
Test set score: 0.8021
```

混淆矩陣

```
Confusion matrix
```

```
[[ 73   0  56   0]  
 [ 20   0   0   0]  
 [ 12   0 385   0]  
 [ 25   0   0   0]]
```

分類報告

	precision	recall	f1-score	support
acc	0.56	0.57	0.56	129
good	0.00	0.00	0.00	20
unacc	0.87	0.97	0.92	397
vgood	0.00	0.00	0.00	25
accuracy			0.80	571
macro avg	0.36	0.38	0.37	571
weighted avg	0.73	0.80	0.77	571

改善後:

使用基尼指數作為準則實例化決策樹分類器模型

```
clf_gini = DecisionTreeClassifier(max_depth=5, min_samples_split=10)
```

使用基尼係數檢定準確率

```
Model accuracy score with criterion gini index: 0.8774
```

比較訓練集和測試集的準確率

```
Training-set accuracy score: 0.8686
```

```
Training set score: 0.8686
```

```
Test set score: 0.8774
```

混淆矩陣:

```
Confusion matrix
```

```
[[ 99   0  23   7]
 [ 11   0   0   9]
 [ 21   0 376   0]
 [  3   0   0  22]]
```

分類報告:

	precision	recall	f1-score	support
acc	0.74	0.77	0.75	129
good	0.00	0.00	0.00	20
unacc	0.94	0.95	0.94	397
vgood	0.58	0.88	0.70	25
accuracy			0.87	571
macro avg	0.57	0.65	0.60	571
weighted avg	0.85	0.87	0.86	571

改善前後對比圖:

	改良前	改良後
Training set	0.7865	0.8686

Test set	0.8021	0.8774
macro avg f1-score	0.37	0.60
weighted avg f1-score	0.77	0.86

第二題 Random Forest Classifier Tutorial:

改良前:

將資料拆分為單獨的訓練集和測試集

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.33,  
random_state = 42)
```

導入隨機森林分類器

```
from sklearn.ensemble import RandomForestClassifier
```

實例化分類器

```
rfc = RandomForestClassifier(random_state=0)
```

訓練模型

```
rfc.fit(X_train, y_train)
```

預測測試集結果

```
y_pred = rfc.predict(X_test)
```

檢查準確率分數

```
from sklearn.metrics import accuracy_score
```

```
print('Model accuracy score with 10 decision-trees : {0:0.4f}'.  
format(accuracy_score(y_test, y_pred)))
```

```
Model accuracy score with 10 decision-trees : 0.9247
```

使用 n_estimators = 100 實例化分類器

```
rfc_100 = RandomForestClassifier(n_estimators=100, random_state=0)
```

將模型擬合到訓練集

```
rfc_100.fit(X_train, y_train)
```

根據測試集結果進行預測

```
y_pred_100 = rfc_100.predict(X_test)
```

檢查準確率分數

```
print('Model accuracy score with 100 decision-trees : {0:0.4f}'.  
format(accuracy_score(y_test, y_pred_100)))
```

```
Model accuracy score with 100 decision-trees : 0.9457
```

混淆矩陣:

```
Confusion matrix
```

```
[[107  8  7  7]  
 [  0 17  1  2]  
 [ 10  0 387  0]  
 [  3  4  0 18]]
```

分類報告:

	precision	recall	f1-score	support
acc	0.89	0.83	0.86	129
good	0.59	0.85	0.69	20
unacc	0.98	0.97	0.98	397
vgood	0.67	0.72	0.69	25
accuracy			0.93	571
macro avg	0.78	0.84	0.81	571
weighted avg	0.93	0.93	0.93	571

改良後:

類別欄位做 One-Hot 編碼（丟掉 class，只編碼特徵）

```
cat_cols = ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety']
```

```
X_onehot = pd.get_dummies(df[cat_cols], drop_first=False)
```

標籤仍用原本的文字類別即可

```
y = df['class']
```

目前資料集的特徵欄位都是類別字串（buying、maint、doors、persons、lug_boot、safety），df.head() 可見像 'low'、'vhigh' 等值，直接餵給模型會出現「could not convert string to float」錯誤，因此需先把 X 做 One-Hot 編碼再訓練。

將資料拆分為單獨的訓練集和測試集

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.33, random_state=42, stratify=y
```

```
)
```

```
# 導入隨機森林分類器

from sklearn.ensemble import RandomForestClassifier

# 實例化分類器

rfc = RandomForestClassifier(

    n_estimators=300,

    max_depth=None,      # 允許充分擬合，由森林投票抑制過擬合

    max_features='sqrt', # 分類任務常用選項

    criterion='entropy',  # 試試資訊增益，有時優於 gini

    n_jobs=-1,           # 用滿 CPU 核心以加速

    random_state=42

)
```

```
Model accuracy score with 10 decision-trees : 0.9440
```

```
# 使用 n_estimators = 300 實例化分類器

rfc_300 = RandomForestClassifier(n_estimators=300, random_state=0)

# 將模型擬合到訓練集

rfc_300.fit(X_train, y_train)

# 根據測試集結果進行預測

y_pred_300 = rfc_300.predict(X_test)

# 檢查準確率分數

print('Model accuracy score with 300 decision-trees : {0:0.4f}'.
      format(accuracy_score(y_test, y_pred_300)))
```

Model accuracy score with 300 decision-trees : 0.9475

混淆矩陣:

Confusion matrix

```
[[108 12  6  3]
 [  0 18  0  2]
 [ 10  0 387  0]
 [  3  2  0 20]]
```

分類報告:

	precision	recall	f1-score	support
acc	0.89	0.84	0.86	129
good	0.56	0.90	0.69	20
unacc	0.98	0.97	0.98	397
vgood	0.80	0.80	0.80	25
accuracy			0.93	571
macro avg	0.81	0.88	0.83	571
weighted avg	0.94	0.93	0.94	571

改善前後對比圖:

	改良前	改良後
10棵樹	0.9247	0.9440
100->300棵樹	0.9457	0.9475
macro avg	0.81	0.83
weighted avg	0.93	0.94

第四題 摘要報告:

一、緒論與決策樹分類器基礎

機器學習與知識獲取的瓶頸

自人工智慧在 1950 年代中期獲得認可以來，機器學習一直是核心研究領域。機器學習之所以重要，一方面是因為學習能力是智慧行為的標誌，另一方面它提供了一種建構高性能系統的潛在方法。早期的研究涵蓋了調整內部參數以提高性能的自適應系統（如 Samuel 的下棋程序），以及將學習視為獲取結構化知識（如概念、歧視網路或生產規則）的方法。

然而，知識型專家系統的出現突顯了知識獲取的巨大挑戰。構建早期專家系統所需的知識，通常是透過領域專家和知識工程師之間耗時的互動來編纂的。這種方法知識闡述的典型速率約為每人天幾條規則，但一個複雜任務的專家系統可能需要數百甚至數千條規則。Feigenbaum (1981) 將此稱為「知識獲取瓶頸」問題。這極大地刺激了研究人員探索機器學習方法，作為一種闡述知識的手段。

決策樹的表示法與優勢

決策樹（Decision Tree）被認為是表示分類器最受歡迎的方法之一。它是一種將實例空間進行遞歸劃分的分類器。

決策樹的結構是一個有根樹：

- 內部節點（Internal Node 或 Test Node）：表示對輸入屬性值的某個離散函數進行的測試，並將實例空間劃分為兩個或多個子空間。在最簡單且最常見的情況下，每個測試只考慮單個屬性。
- 葉節點（Leaves 或 Terminal Nodes）：賦予一個類別名稱，代表最合適的目標值。葉節點也可以包含一個概率向量，指示目標值屬於每個類別的條件概率。
- 分類過程：實例從樹根向下導航，根據沿途測試的結果，直到遇到葉節點，從而確定其實例類別。

決策樹的分類能力幾何上可以解釋為一組超平面，每個超平面都與其中一個軸正交（在單變量測試情況下）

決策樹的主要優勢包括：

1. 易於理解（Comprehensible）：決策樹是自我解釋的，當被壓縮時很容易遵循。它們可以轉換成一組規則，因此這種表示法被認為是可理解的。
2. 通用性（Versatility）：能夠處理名義（nominal）和數字（numeric）輸入屬性。
3. 表示能力：決策樹表示法足夠豐富，可以表示任何離散值的分類器。
4. 魯棒性（Robustness）：能夠處理包含錯誤或缺失值的資料集。
5. 非參數法：決策樹被認為是一種非參數方法，這意味著它們對空間分佈和分類器結構沒有假設。

TDIDT家族概述

TDIDT (Top-Down Induction of Decision Trees, 自上而下歸納決策樹) 家族是一類學習系統的統稱, 它們在學習策略、知識表示和應用領域上具有強烈的共同聯繫。TDIDT 家族的成員皆處理從實例中歸納決策樹的相同任務。

TDIDT 系統的特點如下:

- 應用領域: 不限於特定領域 (如化學或國際象棋), 而是通用目的的系統。它們解決的應用都涉及分類任務。例如, 從症狀診斷疾病、確定國際象棋位置的博弈論價值, 或根據大氣觀測預測風暴的可能性。
- 知識表示: 將獲取的知識表示為決策樹。這是一種相對簡單的知識形式, 儘管其表達能力不如語義網絡, 但仍能生成足以解決具實際意義的困難問題的知識。
- 學習策略: 採用**非增量式 (non-incremental)** 的從實例中學習。系統會獲得一組與分類任務相關的案例, 並從上而下地開發決策樹。與增量方法不同, TDIDT 系統必須在學習的許多階段檢查並重新檢查所有給定的實例。

TDIDT 家族的先驅是 Hunt 的概念學習系統框架 (CLS, 1966)。CLS 旨在構建一個最小化分類成本的決策樹, 並採用類似 minimax 的前瞻策略。

ID3 (Quinlan, 1979) 是從 CLS 發展而來的一系列程序之一。ID3 採用一種**資訊驅動 (information-driven)** 的評估函數, 而非 CLS 的成本驅動前瞻策略。

ID3 的後代包括:

- ACLS (1983): ID3 的推廣, 允許屬性具有不受限制的整數值。
- ASSISTANT (1984): 進一步推廣, 允許屬性具有連續 (實數) 值, 且允許類別形成層次結構。
- 商業衍生品: Expert-Ease (1983)、EXTRAN (1984) 和 RuleMaster (1984) 是 ACLS 的商業衍生品, 它們在底層理論上沒有顯著進展, 但加入了許多使用者友好的創新。

二、決策樹的歸納與建構

歸納任務與 ID3 詳述

歸納任務 (Induction Task) 的基礎是一個對象的集合 (Universe), 這些對象由一組屬性來描述。每個屬性被限制取一組離散、互斥的值。任務是開發一個分類規則 (在此處表示為決策樹), 可以根據屬性值確定任何對象的類別。

如果屬性是足夠的 (adequate), 則總是可以構建一個正確分類訓練集中每個對象的決策樹。歸納的本質在於超越訓練集, 即構建一個不僅能正確分類訓練集對象, 也能分類**未見過 (unseen)** 對象的決策樹。

ID3 是為處理屬性多且訓練集大的任務而設計的系統。它採用迭代的基本結構：

1. 隨機選擇訓練集的一個子集，稱為 窗口（window），並從中形成一個決策樹。
2. 使用該樹對訓練集中的所有其他對象進行分類。
3. 如果該樹對所有對象都給出正確答案，則過程終止。
4. 否則，將一部分分類錯誤的對象添加到窗口中，並繼續該過程。

經驗證據表明，這種迭代方法通常比直接從整個訓練集構建樹更快地找到正確的決策樹。

核心機制：資訊驅動分割準則

TDIDT 方法的關鍵在於如何為任意對象集合 C 形成一個決策樹。如果 C 為空或只包含同一類別的對象，則最簡單的樹就是一個標有該類別的葉節點。否則，必須選擇一個測試 T 來對 C 進行劃分（Divide and Conquer 策略）。

ID3 採用資訊驅動的方法來選擇測試屬性，其核心原則是選擇能帶來最大資訊增益（maximum information gain）的屬性。

資訊論基礎假設：

1. **概率估計**：任何正確的決策樹對 C 中對象的分類比例，與它們在 C 中的比例相同。
2. **期望資訊**：如果 C 包含 p 個 P 類對象和 n 個 N 類對象，則分類所需的期望資訊為：

$$I(p, n) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

期望資訊需求 $E(A)$ ：如果屬性 A 將 C 劃分為子集 $\{C_1, C_2, \dots, C_v\}$ ，其中 C_i 包含 p_i 個 P 類和 n_i 個 N 類對象，則以 A 為根的樹所需的期望資訊是加權平均值：

$$E(A) = \sum_{i=1}^v \frac{p_i + n_i}{p+n} I(p_i, n_i)$$

資訊增益 $Gain(A)$ ：分支到 A 所獲得的資訊為：

$$Gain(A) = I(p, n) - E(A)$$

ID3 選擇使 $Gain(A)$ 最大化的屬性。最大化增益等同於最小化 $E(A)$ ，即屬性 A 與類別之間的互信息（mutual information）。

分割準則的比較與多值屬性問題

資訊增益（Information Gain）是基於雜質（Impurity）的準則。其他常見的雜質準則包括：

- **Gini Index**：衡量目標屬性值概率分佈之間的分歧。
- **Likelihood Ratio Chi-Squared Statistics**：用於衡量資訊增益的統計顯著性。

然而，雜質準則傾向於偏愛具有較大域值的屬性。例如，一個代表國民安全號碼的屬性可能會獲得最高的資訊增益，但對決策樹沒有用處。這促使研究人員尋求歸一化（Normalized）的雜質準則。

增益比率（Gain Ratio）：Quinlan 提出了增益比率來「歸一化」資訊增益：

$$\text{Gain Ratio}(A) = \frac{\text{Information Gain}(A)}{\text{Split Information}(A)}$$

其中 $\text{Split Information}(A)$ 衡量了確定屬性 A 的值本身所需的資訊內容。增益比率準則從那些增益高於平均水準的屬性中，選擇使此比率最大化的屬性。

子集準則（Subset Criterion）：ASSISTANT 系統通過要求所有測試只有兩個結果來解決偏好問題。它選擇一個屬性值的子集 S ，並創建兩個分支：一個用於 S 中的值，另一個用於其餘值。Kononenko 等人報告，此修改導致決策樹更小且分類性能有所改善。然而，它可能導致樹對人類來說更難以理解，並可能大幅增加計算需求，因為一個具有 v 個值的屬性有 $2^v - 1$ 種不同的二元分割方式需要檢查。

經驗比較表明，當所有屬性都是二元時，增益比率準則能產生更小的決策樹。當任務涉及具有大量值的屬性時，子集準則能產生更小且預測性能更好的樹，但計算量大得多。總體而言，增益比率準則在選擇好的樹根屬性方面表現良好。

決策樹的修剪方法 (Pruning)

使用寬鬆的停止準則往往會生成**過度擬合（over-fitted）**訓練集的大型決策樹。修剪方法旨在解決這個困境，將過度擬合的樹修剪回更小的樹，從而提高泛化準確性。修剪的另一個動機是「以準確性換取簡潔性」。

常見的修剪方法包括：

1. **成本複雜度修剪（Cost-Complexity Pruning / Weakest Link Pruning）**：Breiman 等人提出。分兩階段進行：首先，在訓練數據上構建一系列樹；然後，根據對泛化錯誤的估計，選擇其中一棵樹作為修剪後的樹。
2. **減少錯誤修剪（Reduced-Error Pruning, REP）**：Quinlan 提出。從底層向上層遍歷內部節點，檢查用最頻繁的類別替換該節點是否不會降低樹的準確性。如果不會，則修剪該節點。它保證對於給定的修剪集，能得到最小的準確子樹。
3. **悲觀修剪（Pessimistic Pruning）**：Quinlan 提出，避免了對修剪集或交叉驗證的需求。它使用對二項分佈的連續性校正來估計錯誤率，並在錯誤率在一個標準誤範圍內時進行修剪。
4. **基於錯誤的修剪（Error-Based Pruning, EBP）**：C4.5 算法中實現的，是悲觀修剪的演變。它基於統計信賴區間的上界來估計錯誤率，並在底部向上遍歷節點，比較保持原狀、修剪節點或用最頻繁的子樹替換三者的錯誤率，選擇最低者。
5. **最小描述長度修剪（MDL Pruning）**：使用最小描述長度來評估節點的泛化準確性。它通過編碼樹所需的位數來衡量樹的大小，偏好可以用較少位數編碼的樹。

比較研究表明，不同的修剪技術各有優缺，沒有一種方法在任何情況下都能超越其他方法。

三、處理現實世界的挑戰

處理雜訊數據 (Noise)

在基於真實世界數據的歸納任務中，訓練集資訊不太可能完全準確，可能存在測量錯誤、主觀判斷偏差，甚至樣本被錯誤分類的情況，這些非系統性錯誤統稱為**雜訊 (Noise)**。

雜訊可能導致兩個主要問題：

1. **屬性不足 (Inadequate Attributes)**：雜訊可能導致具有相同屬性描述的對象屬於不同的類別，使得屬性無法區分這些對象。
2. **虛假複雜性**：雜訊可能導致決策樹變得過於複雜，試圖去「擬合雜訊」。

為應對雜訊，算法需要兩個修改：

1. **能夠使用不足的屬性**：雜訊可能使最全面的屬性集看起來不足。
2. **決定何時停止測試**：算法必須能夠判斷進一步測試屬性是否不能改善決策樹的預測準確性。

卡方測試 (Chi-Square Test)：一種有效的解決方案是使用卡方測試來評估屬性與類別是否獨立。如果一個屬性的不相關性無法以極高的置信水準（例如 99%）被拒絕，則應阻止對該屬性進行測試。這被發現能有效防止過於複雜的樹。

當所有測試都被排除，但節點 C 仍然包含多個類別的對象時，必須生成一個葉節點。此時，最佳策略是將葉節點分配給**數量較多 (more numerous)** 的類別，因為這能最小化預期絕對錯誤。

實驗表明，低水準的雜訊不會導致樹構建機制崩潰。此外，如果在受控條件下構建的正確決策樹用於分類被破壞的數據，其性能可能不如從被類似水準破壞的數據中形成的**不完美**決策樹。這提示：如果歸納出的樹在實際使用時會受到高雜訊的影響，那麼從訓練集中消除屬性資訊的雜訊可能是適得其反的。

處理未知屬性值 (Unknown Attribute Values)

在訓練集中遇到案例歷史不完整的盟友問題是未知屬性值。有幾種處理策略：

1. **貝葉斯形式化 (Bayesian Formalism)**：ASSISTANT 使用貝葉斯形式化來估計該對象具有某個屬性值 A_i 的概率，方法是檢查該屬性在訓練集中的分佈，並根據類別進行調整。

2. 決策樹方法：使用訓練集中已知 A 值的子集 C' ，構建一個決策樹來預測未知 A 值的對象。
3. 最常見值（Most Common Value）：簡單地用該屬性在訓練集中最常見的值來替換未知值。

然而，這些猜測方法表現並不理想。另一種嘗試是將「未知」視為該屬性的一個新可能值，但這可能導致結果與常識完全相反，例如，屬性具有未知值可能會虛假地增加其資訊增益。

比例分配策略（Proportional Distribution Strategy）：一種有效的策略是：當評估屬性 A 的資訊增益時，將具有未知值的對象按比例分配給 A 的各個已知值。當選擇了屬性後，在形成子集 C_i 之前，具有該屬性未知值的對象會被丟棄。

分類時的處理：當對象在分類時遇到具有未知屬性 A 的測試節點時，程序應探索所有分支。這可以概念化為將一個「權杖（token）」的值按比例分配給每個分支 A_i 。最終，所有葉節點的權杖值會按類別求和，權杖值最高的類別即為分類結果。實驗表明，這種方法隨著未知值發生率的增加，性能下降非常平穩。

大型數據集歸納

隨著數據量快速增長，產生了对能處理大型數據集決策樹歸納算法的需求。早期的加速方法，例如 Catlett 的方法和 Fifield 的並行實現，仍然要求所有數據必須載入到主記憶體中。Chan 和 Stolfo 建議將數據集劃分為多個不相交的子集，然後訓練多個決策樹並將其組合，但實驗結果顯示，分區可能會降低分類性能。

SLIQ (Mehta et al.)：這是一種不需要將整個數據集載入主記憶體的算法，它使用輔助記憶體（磁碟）。然而，它仍有一個上限制，因為它使用了一個與數據集大小成比例的、必須常駐主記憶體的數據結構。

SPRINT (Shafer et al.)：這是一種可擴展的並行分類器，它移除了所有決策樹歸納的記憶體限制。

RainForest (Gehrke et al.)：一個統一的框架，旨在擴展文獻中任何特定的決策樹算法（包括 C4.5, CART, CHAID）。它在 SPRINT 的基礎上實現了三倍的改進。

四、決策森林與隨機子空間法

單樹分類器的限制與多分類器的優勢

單個決策樹分類器雖然具有直觀、訓練簡單和分類速度快的優點，但它們有一個根本的限制：對於固定的訓練數據，它們容易過度適應（overly adapted），即過度擬合訓練數據。雖然修剪可以提高泛化準確性，但往往是以犧牲訓練數據

上的準確性為代價。過去，人們認為沒有任何方法可以在增加樹的複雜性的同時，同時提高訓練集和測試集的準確性。

然而，這種困難並非樹分類器本身固有。組合多個分類器的方法，可以彌補單個分類器的偏差。

隨機子空間法 (Random Subspace Method, RSM)

隨機子空間法 (RSM) 是一種構建多樹分類器（決策森林）的方法，它能在保持訓練數據最高準確性的同時，提高泛化準確性。

核心機制：

1. 選擇子空間：RSM 依賴於一個自主的、偽隨機程序，從給定的 n 維特徵空間中選擇一個小數量的維度。對於 n 維特徵空間，理論上有 2^n 種選擇的可能性。
2. 構建決策樹：在每個選定的子空間中，使用整個訓練集來構建一個決策樹。這些樹被要求完全分割（fully split），確保對訓練數據達到 100% 的準確性（假設沒有內在歧義）。
3. 泛化差異：每棵樹的分類對於僅在未選定維度上與訓練點不同的點來說是不變的（invariant）。因此，每棵樹以不同的方式進行泛化。高維特徵空間中的巨大子空間數量為選擇提供了極多機會。
4. 優勢：RSM 與奧卡姆剃刀原理（Occam's Razor）相反，它通過增加分類器的複雜性（樹的數量），反而改善了泛化準確性。該方法與樹構建算法的細節無關。

RSM 是一種並行學習算法，每棵決策樹的生成是獨立的，這使其適合於並行實現。

組合函數與樹木獨立性

單個樹的決策通過組合函數（Combination Function）平均葉節點上每個類別的條件概率來實現的。

對於一個點 x ，其屬於類別 c 的判別函數 $g_c(x)$ 定義為所有 n_t 棵樹的概率估計平均值：

$$g_c(x) = \frac{1}{n_t} \sum_{j=1}^{n_t} P(c|v_j(x))$$

其中 $P(c | u_j(x))$ 是點 x 落入第 j 棵樹終端節點 $u_j(x)$ 時，屬於類別 c 的概率。對於完全分割的樹，該決定規則等同於多數投票（plurality vote）。該判別函數顯然可以保持訓練集上 100% 的準確性。對於未見過的點， $g(x)$ 通過平均這些鄰域（決策區域）的後驗概率，來近似原始特徵空間中的後驗概率。

獨立性：決策森林要獲得比單個樹更好的準確性，關鍵在於樹之間必須有足夠的獨立性或相異性（independence or dissimilarity）。RSM 方法通過隨機子空間選擇機制，成功地產生了相異性較高的樹。

與訓練集子採樣方法的比較

除了隨機子空間法外，還有其他構建多分類器的方法，例如基於交叉驗證（cross-validation）的訓練集子採樣（subsampling）方法。

- **Bagging (Bootstrapping)**：從原始訓練樣本中獨立且隨機地選擇子集（有放回），然後用每個子集訓練一個分類器。

- **Boosting**：每個子集的創建依賴於先前的分類結果，引入概率分佈來偏好那些先前分類器錯誤的樣本。

實驗比較結果顯示：

1. 對於所有測試的數據集，決策森林的準確性都高於單樹分類器。
2. 雖然訓練集子採樣構建的單個樹有時可能更準確，但隨著樹的增加，隨機子空間森林的組合準確性通常更優越。
3. 在許多數據集中，子空間法產生的樹比其他方法產生的樹相異性更高（樹協定性 $s_{i,j}$ 更低）。
4. RSM 在特徵和樣本數量都很大的數據集上表現最佳，但在特徵數量非常少或類別數量非常多的數據集上則沒有優勢。

這表明，RSM 在數據集中存在冗餘特徵（redundancy in features）時特別有效。

五、決策樹歸納的細節機制與評估標準

分割準則的分類與基礎

決策樹歸納器（Inducer）的目標是構建一個能夠將實例空間（Instance Space）遞歸劃分的分類器。這個過程涉及選擇最佳屬性來進行分割（Splitting Criteria）。這些標準可以根據其結構分為三類：基於雜質的準則（Impurity Based Criteria）、正規化雜質準則（Normalized Impurity Based Criteria）和二元準則（Binary Criteria）。

雜質基礎（Impurity Based Criteria）：雜質衡量的是目標屬性值的概率分佈有多混亂。如果一個隨機變量的概率向量中，某個分量為 1（即變量只取一個值），則該變量被定義為**純淨的（pure）**。相反地，如果所有分量都相等，則雜質水平達到最大。

資訊增益（Information Gain）：資訊增益是基於雜質的準則，使用**熵（Entropy）**作為雜質度量。

Gini 指數 (Gini Index)： Gini 指數也是一種基於雜質的準則，衡量目標屬性值概率分佈之間的分歧程度。Gini 指數定義如下：

$$\text{Gini}(S) = 1 - \sum_{i=1}^m p_i^2$$

選擇屬性的評估標準是 Gini 增益 (GiniGain)，即分割後目標屬性雜質的減少程度。

可能性比率卡方統計量 (Likelihood Ratio Chi-Squared Statistics)：可能性比率 InformationGain/Entropy(Y) 用於衡量資訊增益的統計顯著性。

零假設是輸入屬性與目標屬性是條件獨立的。如果零假設成立，該測試統計量（乘以兩倍）近似於 χ^2 分佈，自由度為 $(v-1)(m-1)$ ，其中 v 是輸入屬性 A 的值數量， m 是目標屬性 Y 的值數量。

克服偏差的正規化準則

傳統的雜質準則（如資訊增益）傾向於**偏愛具有較大定義域值（larger domain values）的屬性**。例如，一個代表國民安全號碼的屬性，雖然獲得的資訊增益最高，但對決策樹毫無用處。因此，需要使用正規化（Normalized）雜質準則。

增益比率 (Gain Ratio)： 增益比率是 Quinlan 提出的，用來**「正規化」資訊增益**。它通過將資訊增益除以屬性本身固有的資訊內容 $IV(A)$ （分裂資訊 SplitInformation）來實現：

$$\text{Gain Ratio}(A) = \frac{\text{Information Gain}(A)}{\text{Split Information}(A)}$$

其中 $\text{SplitInformation}(A)$ 衡量了確定屬性 A 的值本身所需的資訊內容。由於 $IV(A)$ 可能為零，或者可能偏愛 $IV(A)$ 極小的屬性，因此 Quinlan 建議分兩階段選擇：首先計算所有屬性的資訊增益，然後只考慮那些增益高於平均水準的屬性，並從中選擇使增益比率最大化的屬性。

距離測量 (Distance Measure)： Lopez de Mantras 引入了一種距離測量，也屬於正規化雜質準則，但其正規化方式與增益比率不同。

二元與多變量分割準則

二元準則 (Binary Criteria)： 二元準則用於創建二元決策樹（Binary Decision Trees），其基礎是將輸入屬性域劃分為兩個子域。

- **Twoing 準則：** Breiman 等人指出，Gini 指數在多類別問題中可能表現不佳，因此建議使用 Twoing 準則。對於多類別問題，Twoing 準則偏愛均勻劃分的屬性。當目標屬性是二元時，Gini 和 Twoing 準則等效。

- **正交準則 (Orthogonality Criterion, ORT)**：Fayyad 和 Irani 提出的二元準則，它通過計算兩個分佈向量之間的夾角來衡量分割的優劣。

- **Kolmogorov – Smirnov 準則**：Friedman 和 Rounds 提出的二元準則，使用 Kolmogorov – Smirnov 距離。該準則後被 Utgoff 和 Clouse 擴展以處理多類別和缺失數據。

單變量分割準則的比較：許多比較研究表明，在大多數情況下，選擇不同的分割準則對樹的性能影響不大。每個準則在特定情況下表現優越，這符合「沒有免費午餐」定理（No-Free-Lunch Theorem）。

多變量分割準則 (Multivariate Splitting Criteria)：多變量分割準則允許多個屬性參與單個節點的測試。雖然找到最佳多變量準則更複雜，且在文獻中不如單變量準則流行，但它可以顯著提高樹的性能。大多數多變量準則基於輸入屬性的**線性組合 (linear combination)**。尋找最佳線性組合的方法包括貪婪搜索、線性規劃、以及線性判別分析等。

處理缺失值 (Handling Missing Values) 的策略細節

在現實數據集中，缺失值（Missing Values）是很常見的問題，這使得歸納（訓練集有缺失）和分類（新實例有缺失）都變得複雜。

Quinlan 的比例分配法 (Proportional Distribution Strategy)：Quinlan 的方法在**計算分割準則**時，會按比例減少該準則的值，因為系統沒有從這些缺失的實例中學到任何東西：

$$\text{Corrected Criterion}(A) = \text{Criterion}(A) \times \frac{|S| - |S_u|}{|S|}$$

其中 $|S_u|$ 是在 S 中屬性 A 的值缺失的實例數。如果使用了正規化準則（如增益比率），則分母的計算應將缺失值視為屬性域中的一個**額外值**。

當一個節點被分割後，具有缺失屬性值的實例會被分配到每個出射邊，賦予相應的權重： $\text{weight}_i = (|S_i| / (|S| - |S_u|))$ 。在分類新實例時，如果遇到屬性值缺失的測試節點，該實例（的概念性「權杖」）會被傳遞到所有出射邊，並按比例加權。最終，葉節點上所有權杖值將按類別求和，權杖值最高的類別即為分類結果。實驗證明，這種方法隨著未知值發生率的增加，性能下降非常平穩（graceful degradation）。

代理分割 (Surrogate Splits) - CART 的方法：Breiman 等人（CART 算法）提出了代理分割（Surrogate Splits）代理分割，該代理分割使用不同的輸入屬性，但與原始分割**最相似**。如果在原始分割中使用的屬性值缺失，則使用最佳代理分割來決定實例的路徑。

其他方法： Loh 和 Shih 建議通過檢查其他實例來估計缺失值。如果名義屬性值缺失，則使用具有相同目標屬性值的實例中該屬性的**眾數（mode）**均值（mean）。

六、決策樹的修剪方法與主流算法特徵

主要修剪方法的詳細介紹

過於寬鬆的停止準則會導致樹過度擬合；修剪則是在樹成長得過大後，將其削減回更小的樹。修剪的另一個動機是「以準確性換取簡潔性」（Trading accuracy for simplicity）。

1. 成本複雜度修剪 (Cost-Complexity Pruning / Weakest Link Pruning) 這是 Breiman 等人（CART 算法）提出的。該方法分兩階段進行：

- 1. 構建樹序列：**從原始樹 T_0 開始，構建一個樹序列 $T_0, T_1, \dots, T_k$ ，其中 T_k 是根節點樹。
- 2. 選擇最佳樹：**根據對泛化錯誤的估計，從序列中選擇一棵樹作為最終的修剪樹。樹 $T_{\alpha+1}$ 是通過將其前身 T_α 中那些在每個修剪的葉子中，**表觀錯誤率增加幅度最小**的子樹替換為葉節點而獲得的。泛化錯誤通常通過將數據集劃分為訓練集和修剪集來估計，或者使用交叉驗證。

2. 減少錯誤修剪 (Reduced-Error Pruning, REP) Quinlan 提出的簡單修剪過程。它從底層向上層遍歷內部節點，檢查用**最頻繁的類別**替換該節點是否**不會降低**樹在一個單獨的****修剪集（pruning set）****上的準確性。如果是，則修剪該節點。REP 保證能產生相對於給定修剪集，**最小的準確子樹**。

3. 最小錯誤修剪 (Minimum-Error Pruning, MEP) Niblett 和 Bratko 提出的。MEP 進行底層向上層遍歷，比較修剪和不修剪節點時的 L -**概率錯誤率估計**。如果修剪不會增加概率錯誤，則修剪該節點。

4. 悲觀修剪 (Pessimistic Pruning) Quinlan 的悲觀修剪 避免了使用單獨的修剪集或交叉驗證。它使用對二項分佈的連續性校正來估計錯誤率。如果一個內部節點的錯誤率在一個標準誤內（one standard error），則修剪該節點。悲觀修剪是****從頂層向下（top-down）****遍歷內部節點的，如果一個節點被修剪，則其所有後代都會從修剪過程中移除，因此速度相對較快。

5. 基於錯誤的修剪 (Error-Based Pruning, EBP) EBP 是悲觀修剪的演變，並在著名的 **C4.5 算法**中實現。EBP 使用統計信賴區間的**上界**來估計錯誤率。該程序**從底層向上層**遍歷所有節點，並比較三種情況下的錯誤率：

- 保持原始子樹 T 的錯誤率。
- 修剪節點 N 的錯誤率（替換為葉節點）。

3. 用最頻繁的子節點 (maxchild) 替換節點 N 的錯誤率。程序選擇錯誤率最低的操作。

6. **最小描述長度修剪 (Minimum Description Length Pruning, MDL)** MDL 原理用於評估節點的泛化準確性。該方法通過計算編碼樹所需的位數來衡量樹的大小。MDL 偏好那些可以用較少位數編碼的決策樹。

7. **最優修剪 (Optimal Pruning, OPT)** Bratko 和 Bohanec 引入了最優修剪算法 (OPT)，它使用**動態規劃 (dynamic programming)** 目標**(修剪後) 決策樹**的葉子和內部節點數量有關，通常效率更高。

主流 TDIDT 算法的特點比較

算法	開發者	分割準則	修剪方法	屬性類型處理	缺失值處理	特點
ID3	Quinlan	資訊增益	無	僅限名義	無	簡單，基礎算法
C4.5	Quinlan	增益比率	EBP	數字/名義	有（修正增益比率）	ID3 的演進，處理數字與缺失值
CART	Breiman et al.	Twoing 準則/Gini	成本複雜度修剪	數字/名義	有（代理分割）	僅構造二元樹，能處理迴歸樹
CHAID	Kass	卡方檢驗	無	僅限名義（後擴展）	有（視為單獨類別）	基於統計檢驗，多分支分割
QUEST	Loh and Shih	ANOVA/Chi-square	交叉驗證	數字/名義	無	速度快，偏差小，僅生成二元樹

- **CART 的特殊性**：CART（分類與迴歸樹）的特點在於它只**構造二元樹**，即每個內部節點只有兩條出射邊。除了分類，CART 還能夠生成**迴歸樹 (Regression Trees)**，其葉節點預測一個實數值（通常基於最小平方偏差）。

- **CHAID 的統計基礎**：CHAID（卡方自動交互檢測）最初設計用於名義屬性，它使用統計檢驗來選擇最佳屬性。它將屬性值中統計學上差異最小的對合併，然後選擇 p 值最低的屬性進行分割。CHAID 的停止準則之一是最大樹深，且**不執行修剪**。

- **QUEST 的速度與偏差**：QUEST 算法（快速、無偏、高效、統計樹）通過 ANOVA F 檢驗或卡方檢驗計算屬性與目標屬性之間的關聯度，從而選擇分割屬性。它具有**可忽略的偏差 (negligible bias)**，並使用十重交叉驗證進行修剪。

七、決策森林與隨機子空間法的深化分析

隨機子空間法的核心機制與優勢

RSM 通過自主的、偽隨機程序，從 n 維特徵空間中選擇一個較小數量的維度來構建單個決策樹。每棵樹都是在選定的子空間中，使用整個訓練集構建的，並且被要求完全分割（fully split），以確保在訓練數據上達到接近 100% 的準確性。

泛化差異： 每棵樹的分類結果對於那些僅在**未選定維度**上與訓練點不同的點是**不變的（invariant）**。因此，每棵樹以不同的方式進行泛化。高維特徵空間中存在巨大的子空間數量，這使得 RSM 能夠利用高維度的優勢，而不是遭受**維度詛咒（curse of dimensionality）**的困擾。

與奧卡姆剃刀原理的對比： RSM 與奧卡姆剃刀原理（Occam's Razor）**整體複雜性（樹的數量）**，反而改善了泛化準確性，同時保持了訓練數據上的完美準確性。這表明單一分類器的複雜性限制**不是內在固有的**。

斜角決策樹與分裂函數 (Oblique Decision Trees)

雖然 RSM 獨立於特定的樹構建算法，但早期研究中使用的**斜角決策樹（Oblique Decision Trees）**不必與任一特徵軸平行。

Ho 在實驗中使用了兩種簡單的斜角樹構建方法：

1. **中央軸投影 (Central Axis Projection, CAP)：** 該方法尋找連接兩組數據簇中心點（類別平均值）的直線（中央軸）。所有數據點投影到中央軸上，然後沿軸搜索一個垂直於軸的超平面，該超平面能最小化分類錯誤。CAP 速度快，但由於優化粗糙，可能導致樹非常大。
2. **感知器訓練 (Perceptron Training, PER)：** 該方法使用**固定增量感知器訓練算法**來選擇超平面。它尋找一個超平面來分離兩組類別，直到錯誤計數降為零或達到固定的迭代次數。PER 通過迭代優化，導致樹的大小大幅縮小。

樹木獨立性與組合準確度

決策森林的組合準確性，決定性地取決於樹與樹之間是否具有**足夠的獨立性或相異性（dissimilarity）**。

樹協定性 (Tree Agreement $s_{i,j}$)： Ho 定義了樹協定性（Tree Agreement $s_{i,j}$ ）來量化兩棵樹 t_i 和 t_j 之間的相似度：

$$s_{i,j} = \int_R p(x) dx$$

其中 R 是兩棵樹的類別決策 $c_i(x)$ 和 $c_j(x)$ 相同的所有點 x 的區域。在實證中，通常使用測試樣本集來近似估計樹協定性，即計算測試樣本中兩棵樹分類一致的比例。

RSM 的獨立性優勢：實驗結果顯示，在多數數據集上，隨機子空間法產生的樹比基於訓練集子採樣（如 Bagging 和 Boosting）的方法產生的樹相異性更高（ $S_{i,j}$ 更低）。

- 例如，在 “dna” 數據集上，RSM 的平均樹協定性為 0.7540，而 Bagging 為 0.9081，Boosting 為 0.8969。

優化目標的雙重性：森林的準確性受單個樹的準確性和樹之間的協定性兩者影響。這意味著單獨優化任一因素不一定能產生最佳森林。理想情況是尋找具有最低相似性的最佳單個樹。例如，對於 “dna” 數據，雖然集群方法產生的樹具有最低協定性，但它們的單個準確性較低，導致森林準確性不如增益比率分割的森林。

特徵冗餘的重要性：RSM 在特徵和樣本數量都很大，且特徵冗餘（redundancy in features）人工冗餘（如原始特徵的對應和、差、積或布林組合）來擴展特徵向量。

隨機特徵數量 (Number of Random Features) 的影響

在使用 RSM 時，需要確定在每次分割中應選擇多少特徵。實驗結果顯示，對於 “dna” 數據集（180 個特徵），使用一半的特徵（90 個）通常能達到最佳或接近最佳的組合準確性。

樹協定性一般會隨著隨機特徵數量的增加而增加。這表明，當使用的特徵越多，樹之間的差異性越小，這也解釋了為什麼使用一半特徵能獲得最佳準確性，因為它在保持一定單樹準確性的同時，最大化了獨立性。

決策森林的排序優化

樹協定性度量還可以應用於對決策森林中的樹進行**排序**，以便最不相似的樹最先被評估。這有助於在實踐中實現更好的**速度/準確性權衡**，即使用最少的樹來達到既定準確性。實驗表明，按遞增協定性排序樹，可以比未排序的樹更快地提高準確性。

八、總結

此綜合報告已經詳細探討了決策樹歸納和決策森林技術的各個方面。我們從 ID3 的資訊論基礎出發，深入分析了 TDIDT 框架下用於處理真實世界複雜性（如雜訊、缺失值、過度擬合）的各種機制。

報告的關鍵發現包括：

1. 歸納與優化：TDIDT 算法的性能高度依賴於分割準則的選擇，從資訊增益到增益比率和 Gini 指數。正規化準則（如增益比率）旨在解決多值屬性偏好問題。

2. 魯棒性機制：處理雜訊數據的卡方測試，以及處理缺失值的比例分配策略（Quinlan）和 代理分割（CART），是確保決策樹在不完美數據上仍能保持魯棒性的關鍵。
3. 修剪的重要性：從 CART 的成本複雜度修剪到 C4.5 的 EBP，修剪方法對於避免過度擬合並提高泛化準確性至關重要。
4. 決策森林的突破：隨機子空間法（RSM）成功地克服了單一決策樹的複雜性限制。RSM 透過構建獨立性高的多個分類器（決策森林），在保持訓練集完美準確性的同時，單調地提高了泛化準確性。
5. 核心洞察：RSM 的效能與數據集中特徵冗餘的存在密切相關。樹協定性 si,j 的測量表明，隨機子空間能產生比訓練集子採樣方法（Bagging/Boosting）相異性更高的基分類器。